What Is an Exact Memory Footprint Worth to a Scheduler? Measured Footprints and Job Scheduling on Disaggregated Memory

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A scheduler for a cluster with disaggregated memory can pack jobs only as tightly as its knowledge of each job's memory footprint allows. We measure that knowledge for 23 programs at three inputs each, with the full Pin trace as the truth, MemPrint's windowed and source-only estimates, Massif, DHAT, heaptrack, the gperftools heap profiler and sampled RSS, and we replay the measured footprints through DMSched-Sim, a simulator of 32 nodes with local DRAM and a shared 1 TB pool, in which only the footprint oracle changes. Knowing a job's footprint curve rather than its peak is where most of the gain lies: at 80% offered load the exact curve gives a mean response time 24.6× shorter than the user's request and 5.1× shorter than the exact peak, and it needs 5.0 TB of memory for a service level that the exact peak meets with 6.0 TB and the request with 11.0 TB. Among measured estimators, MemPrint's windowed curves are the only ones that beat the exact peak (358 to 391 against 460 minutes), at 6 to 11 times the native run time against 227 times for the full trace. The heap profilers report the right peak for most programs but read GCC's cc1 at 0.14 to 0.16 times its peak and darknet at 1.97 times, so no margin makes them safe without over-reserving every other job. Curves built from DHAT's three recorded points do worse than DHAT's peak under every margin. When the truth is the resident set on wall-clock time, every measured curve, MemPrint's included, needs about 60% headroom because its clock is not wall-clock time, and peaks do better than curves, which shows that a footprint curve is useful to a scheduler only with a faithful time axis.

1 Introduction

A batch scheduler places a job only after it decides how much memory to reserve for it, and on a cluster with disaggregated memory that decision also sets how much of the shared pool the job takes away from other jobs. Disaggregated memory attaches a pool of memory, over CXL or RDMA, that every node can use in addition to its local DRAM [9, 20]. The pool lets a cluster provision less local memory per node, but only if the scheduler can pack jobs tightly, and it can pack them only as tightly as its knowledge of each job's memory allows. Today that knowledge is weak. Peng et al. found that over 90% of LLNL jobs use under 15% of a node's memory [13], and Zacarias et al. showed that a uniform 60% overestimate of memory demand cuts the throughput of a disaggregated system by about 25% and raises the mean response time about five times [21].

Two kinds of knowledge are possible. A *peak* is one number per job, held for the whole run. A *footprint curve* gives the memory the job needs at every point of its run, so a scheduler can let two jobs whose peaks do not overlap share the same memory. The usual sources of either are the user's request, a heap profiler such as Massif [16], DHAT [17], heaptrack [18] or the gperftools heap profiler [7], the resident set size (RSS) of an earlier run, a learned predictor [9], or a monitor

that resizes a job's allocation while it runs [22]. MemPrint [12] adds two more: a footprint curve measured by instrumenting a sample of a run's memory accesses with Pin [11], and a footprint predicted from the program's source code before it runs.

However, the simulators that study scheduling on disaggregated memory either model one run in cycle detail with no scheduler [15, 3, 4, 19], or treat a job's memory as one declared peak with a synthetic error [21, 20]. None feeds the scheduler the footprints that real profilers report, and none compares those with a footprint curve known before the job starts. It is therefore unclear how much better a scheduler does with an exact footprint than with the tools that exist, and which property of the footprint (its peak, its shape over time, or how safely it bounds the truth) the gain comes from.

In this work, we measure every source of footprint knowledge listed above on the same 23 programs and replay the results through one scheduler, with everything but the footprint oracle held fixed. We make the following contributions:

- We build a corpus of footprint curves for 23 programs at three inputs each, from the full Pin trace (the truth), MemPrint's windowed and static estimates, Massif, DHAT, heaptrack, gperftools and sampled RSS, together with what each method costs (Section 4).
- We build DMSched-Sim, a cluster simulator with local memory and a shared pool, an EASY-backfilling scheduler that admits a job only if its whole reservation curve fits, a remote-memory slowdown model and OOM kills (Section 3).
- We show how much of the gain an exact footprint brings comes from knowing the curve rather than the peak, how close each measured estimator gets, how much memory each needs for the same service, and what each estimator's cost buys (Section 5).
- We evaluate curves derived from DHAT, which records no time series, and show why its three recorded points do not substitute for a measured curve (Section 6).

2 Background and related work

Disaggregated memory simulators. DRackSim [15], CXL-ClusterSim [3], CXL-DMSim [4] and CXLMemSim [19] model the timing of accesses to pooled memory for one run, in cycle detail or from a trace, and none of them has a job scheduler. The BSC Slurm simulator extended by Zacarias et al. schedules real job traces on a disaggregated system [20], but it represents a job's memory as one peak, and its study of accuracy multiplies that peak by a synthetic overestimate [21]. Their dynamic provisioning study resizes allocations from usage traces every five minutes [22], which is a reactive monitor rather than knowledge available before the job starts. Boezennec et al. study the allocation of disaggregated memory to jobs with piecewise-constant memory phases [2], CFM schedules jobs against per-job slowdown curves measured offline [1], and Pond predicts the untouched memory of virtual machines with a gradient-boosted model [9]. KS+ predicts the memory of workflow tasks over time and reduces wasted GB-seconds by 51% against peak-only prediction [8], but it has no cluster scheduler or memory pool.

Footprint estimators. Massif samples the heap at up to 100 snapshots by default, on a time axis of instructions or of milliseconds under Valgrind [16], and with `-pages-as-heap` it counts mapped pages instead of heap blocks. DHAT reports the heap at its global peak (t-gmax) and at exit, and the time of the peak, but no time series [17]. heaptrack records every allocation and can export a heap curve [18], and the gperftools heap profiler writes a dump each time the heap grows

by a set amount [7]. All four count requested heap bytes, so they miss memory a program maps itself and count blocks that are allocated but never touched. MemPrint counts the bytes a program actually touches, from a sample of its memory accesses [12]. Its windowed mode watches accesses only in short windows (1% or 5% of the run) and tracks allocations and page residency in between, and its static mode predicts the footprint from the source code with an interpreter that counts the references a build of the program makes.

3 Simulator

DMSched-Sim replays one stream of jobs once per footprint oracle, so that any difference between two runs comes from what the scheduler knows about memory. It is a time-slotted simulator (one slot is one minute) written in Python (`analysis/memprint/sched/`).

System model. The cluster has 32 nodes, each with 32 cores and 128 GB of local DRAM, and one shared pool of 1 TB (one sixth of all memory). A job runs on one node. Its memory is placed in the node's free local memory first, and the rest goes to the pool.

Jobs. A job has an arrival time, a core count, a base runtime (its runtime with all memory local), a true footprint curve $a(p)$ over its progress $p \in [0, 1]$, and a reservation curve $r(p)$ set by the oracle. Curves are defined over progress rather than time, so a job slowed by remote memory stretches its curve in wall-clock time.

Scheduler. The scheduler is first come, first served with EASY backfilling [10] over a window of 32 queued jobs. Each node keeps a timeline of committed local memory and cores, and the pool keeps one of committed memory. A job starts on a node only if, at every slot from now to its projected end, the part of its reservation that does not fit in the node's free local memory fits in the free pool. With a time-varying reservation, two jobs whose peaks do not overlap can therefore share the same memory. When the head of the queue cannot start, the scheduler reserves its earliest feasible start, and a later job may start early only if it fits beside that reservation. Among the nodes that fit, the scheduler picks the one with the least pool use and then the least free local memory left over (best fit).

Remote-memory slowdown. Following the linear models of CFM [1] and Boezennec et al. [2], a job's runtime is stretched by the share of its resident GB-minutes that lie above its local reservation:

$$T = T_{\text{base}}\,(1 + \beta f_{\text{remote}}), \qquad f_{\text{remote}} = \frac{\sum_t \max(0,\, a(t) - \ell(t))}{\sum_t a(t)}, \tag{1}$$

where $\ell(t)$ is the local part of the reservation and $\beta = 0.5$, so a job whose memory is entirely remote runs 1.5 times as long. Because the runtime depends on the placement and the placement depends on the runtime, the two are solved together by fixed-point iteration when the job is admitted. The scheduler ranks candidate nodes on the job's base runtime and then solves the stretch on the best node, falling back to the next node if the stretched job no longer fits.

OOM kills. If the true footprint exceeds the reservation at any slot, the job is killed there, its work is lost, and it is requeued at the head of the queue with its reservation multiplied by 1.5, as a batch system that raises a job's memory limit after an out-of-memory failure would do. A

reactive oracle (Section 3.3) is the exception: its growth past the reservation is granted from free local memory or the pool while either has room, and the job is killed only when neither does.

Metrics. We report the mean response time (completion minus arrival), the mean bounded slowdown (response over $\max(T_{\text{base}}, 10\,\text{min})$), the number of OOM kills, and the reserved and the actually resident memory as shares of all memory. To leave out the warm-up and the drain of a finite stream, every metric is taken over the jobs that arrive between the 10th and the 90th percentile of arrival times, and the memory shares are averaged over that period.

Validation of the simulator. Ten unit tests (`tests/sched/test_sim.py`) check that two jobs with non-overlapping peaks share a node under their curves but run one after the other under their peaks, that an under-reservation is killed where the footprint passes it and requeued, that backfilling never delays the head of the queue, that a fully remote job runs $1 + \beta$ times as long, and that elastic growth is granted only from free memory. On the synthetic stream of the earlier prototype (3,000 jobs, five footprint shapes, a 60 GB median heap) the simulator reproduces the prototype's ordering at saturation: the exact curve sustains 101 jobs per hour, the exact peak 16% fewer, a Massif-style curve 37% fewer and a DHAT-style peak 57% fewer.

3.1 Job streams

The main experiments use a controlled stream of 3,000 jobs in which memory is the contended resource. Each job has 1 to 16 cores, a lognormal base runtime with a median of 30 minutes clipped to 5 to 720 minutes, and a lognormal memory request with a median of 100 GB. What a job really uses is its request multiplied by a ratio $u \leq 1$ taken from the LANL CM5 log of the Parallel Workloads Archive [5], the archive log that records both requested and used memory (median $u = 0.79$ over its 94,335 jobs with both fields). The shape of the job's footprint, and what every estimator reports for it, come from one corpus program at its largest input (Section 4), chosen at random. All of that program's curves are scaled by the factor that puts the truth's peak at the job's peak, so each estimator keeps its own over- or under-estimate. Arrivals are spread evenly in random order over a span set so that the *offered load*, the resident GB-minutes arriving per minute over the cluster's total memory, takes the values 0.4, 0.6, 0.8, 1.0 and 1.2. Each point is three seeds.

We also tried the MetaCentrum2 log [5] (2013 to 2015, 5.7 million jobs with a memory request but no used memory). In a window of 3,000 single-node jobs, 1% of the jobs carry 92% of the requested GB-minutes, so the results of a replay depend on a handful of very large jobs. We therefore use the controlled stream for the main results.

3.2 Two truths

MemPrint counts the bytes a program touches, but a kernel charges and OOM-kills a process for its resident pages. The two differ by page granularity, by memory an allocator keeps after a free, and by the runtime's own pages (loader, libraries, MPI, an interpreter). We therefore run every experiment with two truths. With *bytes touched* as the truth, the scheduler must cover the footprint a page-perfect system would need. With *RSS* as the truth, it must cover the resident set sampled every 5 ms. The runtime's own pages do not grow with a job's data, so under the RSS truth only the RSS above the run's smallest sample (1.2 to 9.5 MB across the corpus) is scaled to the job's size.

Table 1: Footprint oracles. "Curve" oracles follow a measured footprint over the job's progress; "peak" oracles hold one value for the whole run.

Oracle	Kind	Source
Exact curve	curve	the truth, at full resolution (the upper bound)
Exact, 20 windows	curve	the truth, coarsened as the measured curves are
Exact peak	peak	the truth's peak
MemPrint windowed 1%, 5%	curve	MemPrint watching 1% or 5% of the run
MemPrint pages only	curve	MemPrint page residency, nothing watched
MemPrint static	curve	predicted from source before any run
Massif curve, pages-as-heap	curve	Massif snapshots (heap, or mapped pages), instruction clock
Massif, ms clock	curve	the same, with time in milliseconds under Valgrind
heaptrack curve	curve	heaptrack's heap over time
DHAT peak	peak	heap at DHAT's global peak (t-gmax)
DHAT curve, hold-then-decay	curve	three points recorded by DHAT (Section 6)
gperftools peak	peak	largest in-use heap of the heap profiler's dumps
RSS peak	peak	peak RSS of an earlier run of the same input
User request	peak	the job's memory request
ML peak	peak	request $\times$ a 0.95-quantile gradient-boosted prediction of u
Reactive resizing	curve	request for 5 minutes, then the last 5 minutes' largest use + headroom

3.3 Oracles and margins

An oracle turns what is known about a job into its reservation curve. Table 1 lists them. Curve oracles follow a measured curve, coarsened to 20 windows over progress, each holding its largest value and widened to its neighbours, because each method's clock (memory references for MemPrint, instructions or milliseconds under Valgrind for Massif, seconds for RSS) maps to wall-clock progress only approximately. Peak oracles hold one value for the whole run.

The ML oracle follows Pond [9]. It predicts u from the request, the core count, the requested time and the user's and the executable's median u in their history, with a gradient-boosted model at the 0.95 quantile, trained on half of the CM5 users. The jobs it predicts take their u from the other half of the users. The reactive oracle follows the dynamic provisioning of Zacarias et al. [22]. The scheduler is shown its future resizes when it admits the job, which favours this baseline.

Every estimate needs a margin, because an estimate below the truth causes OOM kills. A curve oracle's margin is headroom added as a share of its own peak, since a curve on another clock can be low where the truth is not, and no factor lifts a zero. A peak oracle's margin multiplies its peak. We use four margin policies: a fixed 10% and 30%, and two *calibrated* margins that a site could set from its own history. For each oracle and program we compute the smallest margin with which the oracle never falls below the truth. A job then receives the 90th percentile (*calibrated 90%*) or the largest value (*calibrated 100%*) of the other programs' margins, so no oracle is tuned on the program it predicts. The request, ML and reactive oracles have no profile to calibrate, and they keep a 10% margin.

4 Footprint corpus

We ran 23 programs at three inputs each, from the smallest to the largest, on a 32-core node with 62 GB of memory (Table 2): four PolyBench kernels [14] (2mm, gemm, jacobi-2d, atax, unoptimised builds at SMALL, MEDIUM and LARGE), miniVite [6] with four threads, the GAP benchmarks bfs, pr, cc and sssp on Kronecker graphs of 2^{16} to 2^{18} vertices, Lua, Python, Perl and SQLite

Table 2: The corpus at each program's largest input: the true peak (bytes touched), the mean over the run as a share of the peak, each method's peak as a share of the true peak, and the time each method takes over the native run.

		Footprint (ratios to the true peak)								Cost (time / native)			
Program	Input	Peak (MB)	Mean/peak	RSS	MemPrint W5	Massif	DHAT	heaptrack	Native (s)	W5	Massif	DHAT	Full trace
2mm	LARGE	37.1	0.96	1.03	1.00	1.00	1.00	1.00	8.6	4.8	5.6	15.2	357
atax	LARGE	32.1	0.87	1.04	1.00	1.00	1.00	1.00	0.1	21.8	5.7	9.8	122
GCC cc1	large	105.5	0.69	1.27	1.06	0.16	0.15	0.15	1.8	100.5	11.0	68.1	206
darknet	alexnet	269.9	0.86	1.07	1.04	1.97	1.97	1.97	3.0	11.8	10.5	45.9	773
GAP bfs	18	72.8	0.36	1.05	1.00	0.99	0.99	0.99	1.5	8.1	3.0	5.7	83
GAP cc	18	72.0	0.36	1.06	1.01	1.01	1.01	1.01	2.2	5.5	2.2	3.9	56
GAP pr	18	72.4	0.40	1.05	1.01	1.00	1.00	1.00	2.8	5.2	2.3	5.3	123
GAP sssp	18	135.8	0.36	0.95	1.00	1.00	1.00	1.00	2.5	5.9	2.4	4.4	116
gemm	LARGE	29.1	1.00	1.04	1.00	1.00	1.00	1.00	6.1	4.4	5.7	15.2	356
HPCCG	64	102.7	0.99	1.03	1.00	1.02	1.02	1.02	1.7	13.3	10.5	24.8	310
jacobi-2d	LARGE	27.1	1.00	1.04	1.00	1.00	1.00	1.00	12.8	3.7	7.3	16.2	275
Kripke	16	274.6	0.88	1.02	1.00	1.00	1.00	1.00	3.4	11.4	4.8	50.7	473
Lua	1	57.1	0.53	1.15	0.94	1.02	0.92	0.92	4.2	14.8	4.3	28.0	161
LULESH	60	151.1	0.62	1.02	1.00	1.00	1.00	1.00	5.5	10.6	7.9	32.2	458
miniVite	32768	15.4	0.51	1.99	1.00	1.55	1.53	1.42	1.8	42.5	22.9	36.9	322
MiniFE	80	256.5	0.98	1.04	1.00	1.03	1.02	1.02	9.2	10.7	6.7	23.8	273
NPB CG	A	27.4	1.00	1.10	1.00	0.99	0.99	0.99	1.9	10.6	7.8	15.5	237
NPB IS	A	69.7	0.81	1.03	1.00	0.99	0.99	0.99	2.4	3.9	2.7	7.0	61
NPB MG	W	58.5	0.98	1.04	1.00	0.99	0.99	0.99	0.5	15.5	4.0	8.8	107
Perl	1	56.1	0.67	1.15	0.97	1.04	0.97	0.96	1.8	30.9	5.2	44.8	145
Python	1	95.9	0.37	1.19	1.04	1.29	1.01	0.14	5.6	17.0	12.5	175.5	272
Quicksilver	100000	42.2	1.00	1.28	1.01	1.02	1.01	1.01	5.5	13.9	6.3	44.5	227
SQLite	1	77.8	0.43	1.06	0.97	1.01	1.01	1.01	1.2	47.7	7.1	39.0	204

workloads at three sizes, GCC's cc1 compiling three source files at `-O0`, HPCCG, LULESH, NPB IS, CG and MG (classes S, W and A), MiniFE, Quicksilver, Kripke, and darknet classifying images with AlexNet (one input). The native runs take 0.12 to 12.8 s, and the true peaks at the largest inputs are 15 to 275 MB.

Every (program, input) ran under ten methods: the program alone, with its RSS sampled every 5 ms; the full Pin trace (MemPrint's splitter with frees tracked), which gives the bytes touched over time and is the truth; MemPrint windowed at 1% and 5% watched and pages only, with the tool built from the `windowed-next` branch; Massif and Massif with `-pages-as-heap` (Valgrind 3.22, up to 1,000 snapshots), each run twice, with Massif's default clock of instructions executed and with milliseconds under Valgrind; DHAT; heaptrack 1.3.0; and the gperftools 2.18 heap profiler with a dump every 1/50 of the native peak RSS. The MemPrint static predictions come from the C interpreter for PolyBench (a curve over time) and from the C++ interpreter for HPCCG, LULESH and NPB (a whole-run footprint). The NPB predictions are those of the second blind test, made before any trace of NPB existed. Of the 802 runs, one failed: the full trace of NPB MG class A ran out of memory, as in the earlier blind test, so MG is used up to class W. The runs shared the node with up to 15 others, so the costs below are slowdowns under load.

4.1 Accuracy of the peak

Most heap profilers report the right peak for most of these programs, and fail badly on the rest (Figure 1). Specifically, Massif's peak is within 5% of the true peak on 19 of the 23 programs, DHAT and gperftools on 19 and heaptrack on 18, because these programs touch nearly all the heap they allocate. However, the heap profilers read 0.14 to 0.16 times the peak of GCC's cc1, whose garbage collector maps its own pages, 1.97 times the peak of darknet and 1.53 to 1.55 times that of miniVite, which allocate memory they never touch. MemPrint windowed at 5% is within 5% on 21 of the 23 programs (0.94 to 1.06 times the truth), and pages only on 21 (0.92 to 1.05). Massif with `-pages-as-heap` overestimates every program (1.07 to 29.9 times), and the RSS is 0.95 to

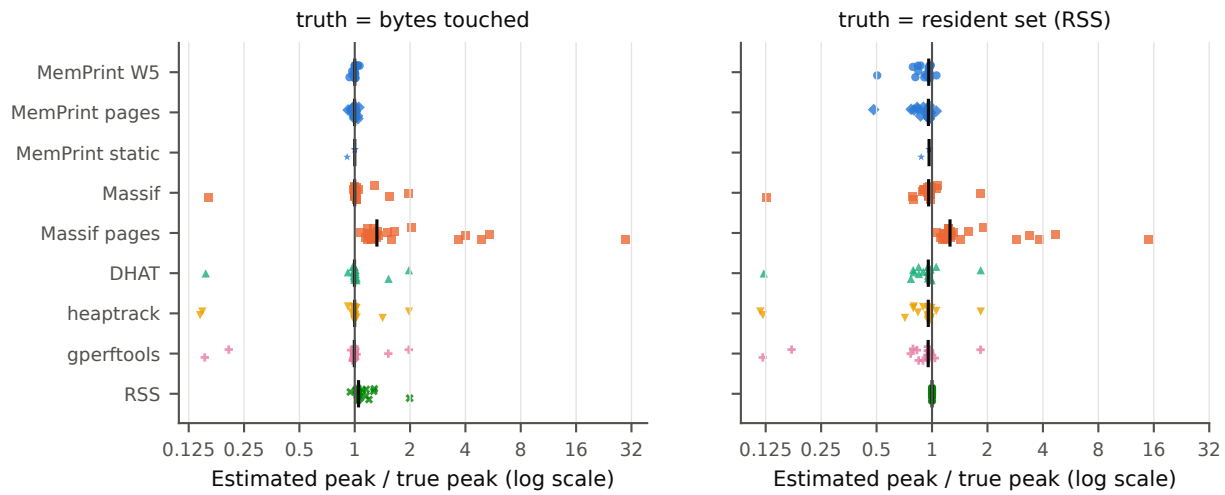


Figure 1: Each method's peak over the true peak, every program at its largest input (one marker per program, the black bar is the median). Left: against bytes touched. Right: against the RSS.

1.99 times the bytes touched (median 1.05).

4.2 Shape over time

The footprint of half of the programs varies over the run. The mean over the run is below 70% of the peak for 11 of the 23 programs (GCC's cc1, the four GAP kernels, Lua, LULESH, miniVite, Perl, Python and SQLite), and it is 0.36 to 0.40 times the peak for the four GAP kernels. A scheduler that holds the peak for such a job reserves memory that the job leaves unused for most of its run. Figure 2 shows Python's footprint, which rises and falls seven times. MemPrint windowed at 5% follows it within a few percent. Massif's curve, on its clock of instructions, follows its phases but sits above it, because Massif counts requested heap bytes and Python allocates memory it never touches (1.29 times the true peak). DHAT records only the peak, its time and the heap at exit, so a curve built from those points misses every phase but one.

4.3 Cost

MemPrint's windowed estimates are not cheaper than the heap profilers on these short runs (Figure 3). At the largest inputs, the median slowdown is 1.4× for heaptrack, 1.2× for gperftools, 5.7× for Massif (6.4× with its clock in milliseconds) and 23.8× for DHAT, against 6.3× for MemPrint pages only, 8.9× at 1% watched and 11.4× at 5% watched. The full Pin trace costs 226.8× (56.1 to 772.5×). Pin's fixed start-up cost weighs heavily on runs of a few seconds, and the windowed mode is slowest on programs with much code to re-translate (GCC's cc1, 93.6× at 1%). The static route needs no run of the program, but its interpreter takes 7 to 59,000 times the native run. It predicts atax MEDIUM in 0.8 s, the other PolyBench kernels at MEDIUM in 13 to 24 s and HPCCG in 255 to 989 s, but NPB IS and CG at class W take 4.0 and 5.5 hours.

4.4 Static predictions

The static predictions are within 0.46% of the true peak at the median, over the 19 (program, input) pairs they cover. They fail in two known ways. For LULESH, the static route predicts

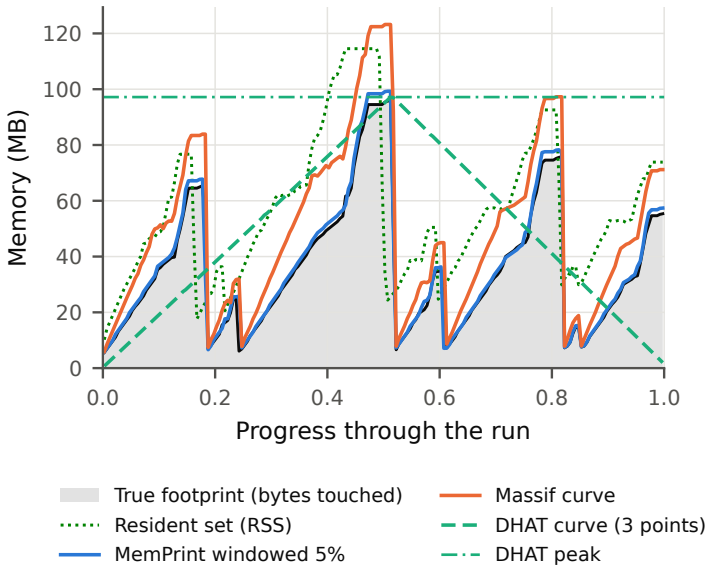


Figure 2: Python (largest input): the true footprint over the run, the RSS, and the curves and peaks the estimators report. Each curve is on its own method's clock, rescaled to progress.

the footprint of the whole run, 1.57 times the live peak, because LULESH frees and reallocates its arrays between iterations and the static route does not model frees for C++ programs. For NPB MG, the prediction is 7 to 9% short, as it was in the blind test, for a reason we have not found.

5 Results

We first ask how much an exact footprint is worth and where the gain comes from, then how close each measured estimator gets, and finally what each estimator's cost buys. Table 3 gives every oracle at offered load 0.8 with calibrated 90% margins, and Figure 4 gives the response time across loads.

5.1 An exact footprint curve

Knowing a job's footprint curve, rather than its peak, is where most of the gain lies. At load 0.8 the exact curve gives a mean response time of 90 minutes, against 460 minutes for the exact peak and 2,215 minutes for the user's request, under the bytes-touched truth, which makes the curve 24.6× better than the request and the peak 4.8× better. The RSS truth gives the same picture (91, 436 and 2,197 minutes). The curve lets the scheduler keep 79% of all memory resident with jobs, against 58% for the exact peak and 27% for the request, because jobs whose peaks fall at different times share the memory that a peak would hold idle. Coarsening the exact curve to the 20 widened windows used for every measured curve raises the response to 158 minutes, so about four fifths of the curve's advantage over the peak survives the resolution and the timing slack a measured curve needs. The gap widens with load (Figure 4). At load 0.4 every oracle that does not over-reserve keeps up with arrivals, and from load 0.6 the peak and the request saturate memory while the curve does not.

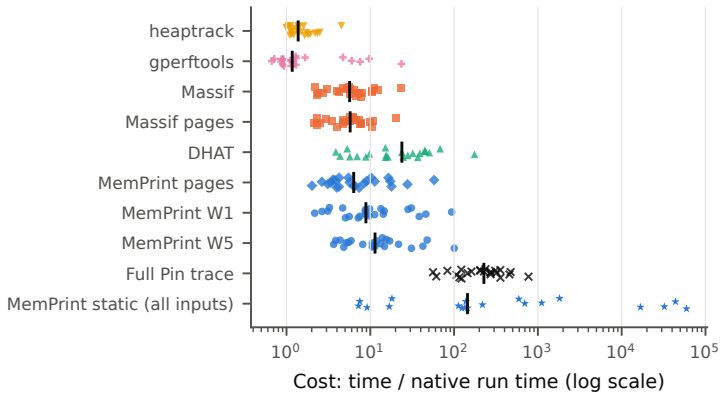


Figure 3: Each method's time over the native run, every program at its largest input (static: every input with a prediction). The black bar is the median.

5.2 Measured estimators

MemPrint's windowed curves are the only measured estimates that beat the exact peak, and only under the bytes-touched truth. With calibrated 90% margins, MemPrint windowed at 1% gives 358 minutes, at 5% 391 minutes and pages only 381 minutes, against 460 minutes for the exact peak, 589 minutes for reactive resizing, 930 to 979 minutes for the RSS, gperftools and DHAT peaks, and 1,077 minutes for heaptrack's curve. However, the MemPrint curves still lose to the exact curve by a factor of four, for two reasons. First, the 20-window coarsening alone costs a factor of 1.8 (from 90 to 158 minutes). Second, a calibrated 90% margin leaves the programs that need more than the 90th-percentile margin under-reserved, which causes 375 to 391 OOM kills per stream. With a fixed 10% margin, MemPrint pages only has no kills and gives 370 minutes.

Massif reports the heap over the whole run, not only its peak, and its curve follows a program's phases on its default clock of instructions executed. With calibrated 90% margins it gives 2,175 minutes, close to the user's request, because the headroom that covers 90% of the programs is 48% of its peak (Table 4). Two kinds of program set that headroom. Massif reads GCC's cc1 at 0.16 times its peak, because the compiler's garbage collector maps its own pages, and on the four GAP kernels its clock of instructions drifts from the truth's clock of memory references enough to need 12 to 53% headroom. With a fixed 10% margin, which leaves those jobs to be killed (1,242 kills per stream), Massif gives 875 minutes, between heaptrack and DHAT. The clock matters. With Massif's clock in milliseconds under Valgrind, which includes Valgrind's start-up and runs at a different pace from the native program, the headroom rises to 84% and the response time to 3,973 minutes. On atax, for example, the heap reaches a tenth of its peak at 48% of Massif's millisecond clock but at the start of the native run. Massif with `-pages-as-heap` counts mapped pages, 1.32 times the true peak at the median, and gives 4,594 minutes. The heap profilers' peaks (DHAT, gperftools) are within 5% of the true peak for 19 of the 23 programs, so they come close to the exact peak for most jobs, but they read GCC's cc1 at 0.14 to 0.16 times its true peak, and its jobs are killed and restarted repeatedly (912 to 1,308 kills per stream). The ML peak (2,006 minutes) improves little on the request it scales, because its 0.95-quantile prediction of u covers only 76% of jobs from unseen users. Reactive resizing (589 minutes) is the best oracle that needs no profile, although it is shown its future resizes when the job is admitted.

Table 3: Every oracle at offered load 0.8, calibrated 90% margins (10% for the request, ML and reactive oracles, which have no profile to calibrate), mean of three seeds. Resp. is the mean response time, BSLD the mean bounded slowdown, Kills the OOM kills per stream of 3,000 jobs, Res. and Used the memory reserved and actually resident as a share of all memory.

	Truth = bytes touched					Truth = RSS				
Oracle	Resp. (min)	BSLD	Kills	Res. %	Used %	Resp. (min)	BSLD	Kills	Res. %	Used %
Exact curve	90	2.7	0	79	79	91	2.7	0	80	80
Exact, 20 windows	158	5.6	0	81	72	168	6.0	0	80	72
MemPrint windowed 1%	358	14.2	375	81	63	1814	78.2	330	59	33
MemPrint pages only	381	15.3	377	79	63	1811	78.0	341	59	33
MemPrint windowed 5%	391	15.7	391	77	61	1807	77.9	316	59	33
Exact peak	460	18.8	0	78	58	436	17.8	0	78	60
Reactive resizing (5 min)	589	24.6	20	71	54	630	26.4	20	70	54
MemPrint windowed 5%, peak	832	34.9	391	73	48	1038	44.0	513	69	45
RSS peak (earlier run)	930	39.4	382	69	44	436	17.8	0	78	60
gperftools peak	955	40.3	1308	70	44	1281	54.6	1309	66	41
DHAT peak	979	41.4	912	70	45	1277	54.4	921	66	40
heaptrack curve	1077	45.6	1512	67	41	3075	133.3	1668	49	23
ML peak (quantile GBM)	2006	86.6	115	61	29	1971	85.2	115	61	30
Massif curve	2175	94.0	729	56	27	2408	104.0	1027	55	29
User request	2215	95.8	0	57	27	2197	95.1	0	57	28
DHAT curve (3 points)	3443	149.7	708	51	22	3074	133.5	806	51	24
Massif curve, ms clock	3973	173.5	1827	44	15	2554	110.4	1026	53	26
DHAT hold-then-decay	4342	188.7	708	48	20	3983	172.8	806	48	21
Massif pages-as-heap	4594	200.1	617	42	16	4452	193.7	643	41	16
Massif pages-as-heap, ms clock	6777	296.1	384	36	11	5889	256.8	759	38	12

5.3 The RSS truth and the time axis

Under the RSS truth, the measured curves lose their advantage. MemPrint's windowed curves give 1,807 to 1,814 minutes, worse than the DHAT peak (1,277 minutes) and the MemPrint 5% peak (1,038 minutes). The reason is the margin, not the shape. MemPrint's curve is on a clock of memory references, while the RSS truth is on wall-clock seconds, so a phase falls at a slightly different point of progress on the two curves, and MemPrint counts bytes rather than pages, which is half of miniVite's RSS. The margin that covers 90% of the programs is therefore 58% of the peak, against 2% under the bytes-touched truth (Table 4), and a curve with 58% headroom on its peak reserves more than a peak. A curve oracle is thus only as useful as its time axis is faithful to wall-clock time. Under the bytes-touched truth the MemPrint curves share the truth's clock of memory references, which favours them, and the RSS truth is the closer model of a real deployment. MemPrint does not record the native time of each snapshot today, and whether doing so closes the gap is left for future work.

5.4 Margins

No margin policy rescues an estimator that is badly wrong for some programs (Figure 5). A small margin lets those programs' jobs be killed, and a margin large enough to cover them over-reserves every other job. With calibrated 100% margins, every heap profiler must cover GCC's cc1, whose peak it reads at 0.14 to 0.16, so each job reserves about 6.5 times its estimate and the response time rises to about 16,000 minutes. The MemPrint curves need at most 7 to 19% under the bytes-touched truth, and their response time with calibrated 100% margins is 362 to 563 minutes. Taking each oracle at its best margin policy changes the order little (Figure 10 in the appendix): under the bytes-touched truth, MemPrint windowed at 5% gives 362 minutes, heaptrack 766, Massif 875 (1,897 with its millisecond clock) and DHAT 979.

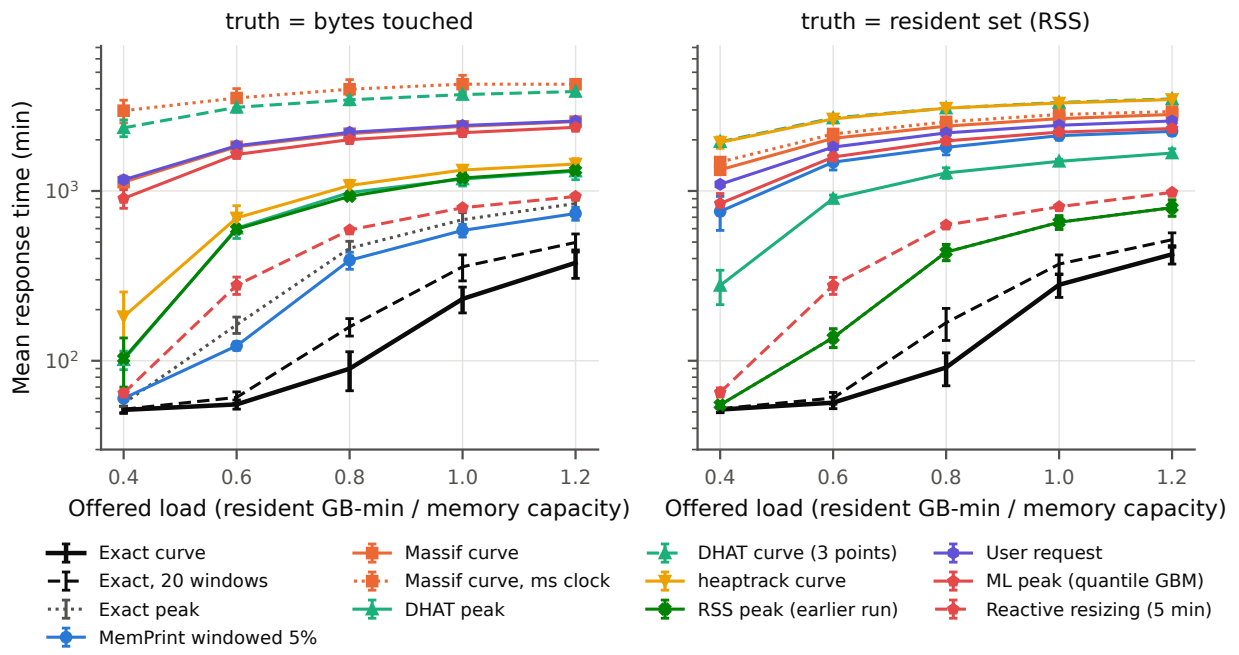


Figure 4: Mean response time against offered load, calibrated 90% margins (95% confidence intervals over three seeds, log scale). Left: the truth is bytes touched. Right: the truth is the RSS.

5.5 Footprints predicted from source

The static route predicts nine of the programs (PolyBench 2mm, gemm and jacobi-2d at MEDIUM and atax at LARGE, HPCCG 64, LULESH 30 and NPB IS, CG and MG at class W), and we replay a stream made of those programs alone (Figure 6). Their footprints are flatter than those of the full corpus, so the exact peak comes close to the exact curve (273 against 194 minutes at load 0.8 under the bytes-touched truth). With calibrated 90% margins, the static predictions give 642 minutes, against 471 minutes for MemPrint windowed at 5%, 623 for heaptrack, 642 for Massif, 662 for the DHAT peak and 2,038 for the request, a 3.2× gain over the request without running the program. Two predictions cause most of the static oracle's 343 kills per stream: NPB MG is 7 to 9% short, and the margin calibrated on the other eight programs is 0%, which does not cover it. LULESH is over-predicted by 1.57 times, which wastes memory rather than causing kills. Under the RSS truth the static curve, like the windowed one, needs more headroom, and it gives 1,709 minutes against 1,399 for the DHAT peak.

5.6 Memory needed for the same service

The response-time gains translate into memory a site would have to buy (Figure 7). At a fixed arrival rate, the exact curve meets the target with 128 GB per node (5.0 TB in all). Under the bytes-touched truth, MemPrint's windowed and pages-only curves and the exact peak need 160 GB per node (6.0 TB, 20% more), reactive resizing and an earlier run's RSS 192 GB (7.0 TB, 40% more), DHAT, gperftools and heaptrack 256 GB (9.0 TB, 80% more), and Massif's curve, the request and the ML peak 320 GB (11.0 TB, 120% more). Massif's curve on its millisecond clock and the DHAT curve do not meet the target even at 320 GB per node. Under the RSS truth, the exact peak and the RSS of an earlier run need 6.0 TB, reactive resizing 7.0 TB, and MemPrint's curves, the heap profilers' peaks, the ML peak and the request 9.0 TB, which repeats the finding that a curve on a

Table 4: The margin each oracle needs to stay above the truth, as a share of its own peak (%), over the 23 programs at their largest inputs.

	Truth = bytes touched			Truth = RSS		
Oracle	Median	90th pct.	Max	Median	90th pct.	Max
MemPrint windowed 1%	0	5	19	4	58	114
MemPrint windowed 5%	0	2	7	4	58	112
MemPrint pages only	0	4	9	4	58	121
Massif curve	0	48	534	2	56	689
Massif pages-as-heap	0	28	62	0	31	63
Massif curve, ms clock	0	84	535	20	63	690
Massif pages-as-heap, ms clock	0	66	78	0	57	76
heaptrack curve	1	22	593	1	75	747
DHAT peak	0	2	546	1	23	706
DHAT curve (3 points)	36	90	580	52	94	739
DHAT hold-then-decay	21	90	580	39	94	739
gperftools peak	0	4	554	2	25	716
RSS peak (earlier run)	0	1	7	0	0	0

reference clock loses its advantage against a wall-clock truth.

5.7 What each estimator's cost buys

Figure 8 sets each estimator's cost against what it buys. Under the bytes-touched truth, the full Pin trace costs 227 times the native run and gives a 24.6× gain over the request as a curve, or 4.8× as a peak. MemPrint's windowed and pages-only curves cost 6 to 11 times the native run and give about 6×, which is a quarter of the exact curve's gain at a twentieth of its cost or less. heaptrack and gperftools cost under 1.5× and give 2 to 3×, DHAT costs 24× and gives 2.3×, and Massif's curve costs 5.7× and gives 2.5× (1.2× with its millisecond clock). An earlier run's RSS costs nothing extra and gives 2.4× under the bytes-touched truth, and under the RSS truth it is the truth's peak. Under the RSS truth the measured curves give about 2×, the same as the heap profilers' peaks, so the profiling cost buys little until the curve's time axis is fixed.

6 DHAT curves

DHAT records the heap at its global peak (t-gmax), the time of that peak, and the heap at exit, but no time series [17]. A scheduler can use those three points in two ways besides the peak. The *DHAT curve* rises linearly from zero to the peak at its recorded time and falls linearly to the exit value. The *hold-then-decay* curve holds the peak from the start to its recorded time, which is safe before the peak, and falls linearly afterwards. Only "never above the peak" is guaranteed by what DHAT records, and that is the DHAT peak oracle.

Both DHAT curves do worse than the DHAT peak under every margin policy and both truths (Figure 9). At load 0.8 with calibrated 90% margins, the DHAT curve gives 3,443 minutes and the hold-then-decay curve 4,342 minutes, against 979 minutes for the DHAT peak, under the bytes-touched truth, and 3,074, 3,983 and 1,277 minutes under the RSS truth. With fixed margins the curves are worse still (7,460 and 9,417 minutes at 10%), because they fall below the truth wherever

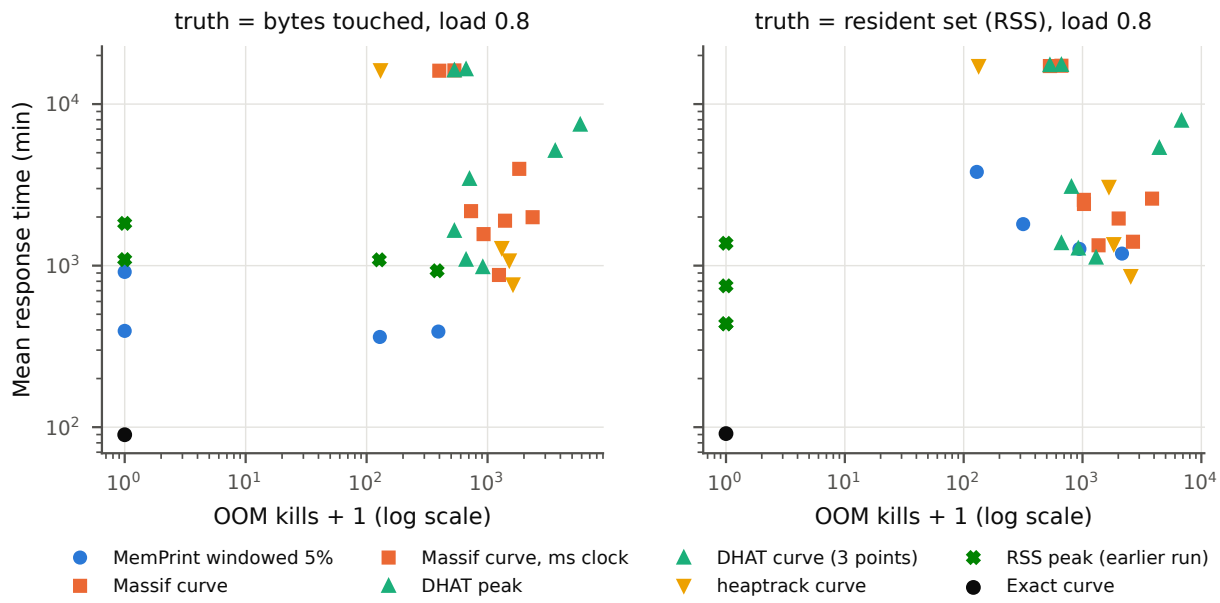


Figure 5: Mean response time against OOM kills at load 0.8, one marker per margin policy (10%, 30%, calibrated 90% and 100%). No margin gives a measured estimator both few kills and a short response time, except the MemPrint curves under the bytes-touched truth.

a program has more than one phase, and their jobs are killed 5,100 to 5,836 times per stream. The median program needs 36% headroom on the DHAT curve (21% on hold-then-decay) and the 90th percentile needs 90% (Table 4). The average of the DHAT curve over the run is 0.87 times the true mean footprint at the median, so it under-reserves on average as well as in places. DHAT's three points therefore do not substitute for a measured curve. The information that makes a curve useful to a scheduler is the timing of every phase, and DHAT records the timing of one.

7 Limitations

The results rest on a model of a cluster and of its jobs, and each simplification limits what they show.

Scaled footprints. The corpus programs peak at 15 to 275 MB, and the simulated jobs at tens to hundreds of GB. Scaling a measured curve to a job's size keeps its shape and every estimator's ratio to the truth, but it assumes the ratio does not change with the input. The corpus shows that it often does at small inputs: MemPrint windowed at 5% reads 0.61 to 0.92 times the peak of the PolyBench kernels at SMALL, and 1.00 at LARGE. We use the largest input of every program to keep this effect small.

Time axes. Every curve is put on the job's progress by linear rescaling of its own clock. The true footprint of a job over wall-clock time is known only for the RSS truth, so the bytes-touched truth favours the MemPrint curves, which share its clock of memory references (Section 5).

The job stream. The main stream is synthetic, with memory the contended resource by design, and the used-to-requested ratio comes from the LANL CM5 log of 1994 to 1996, the only archive

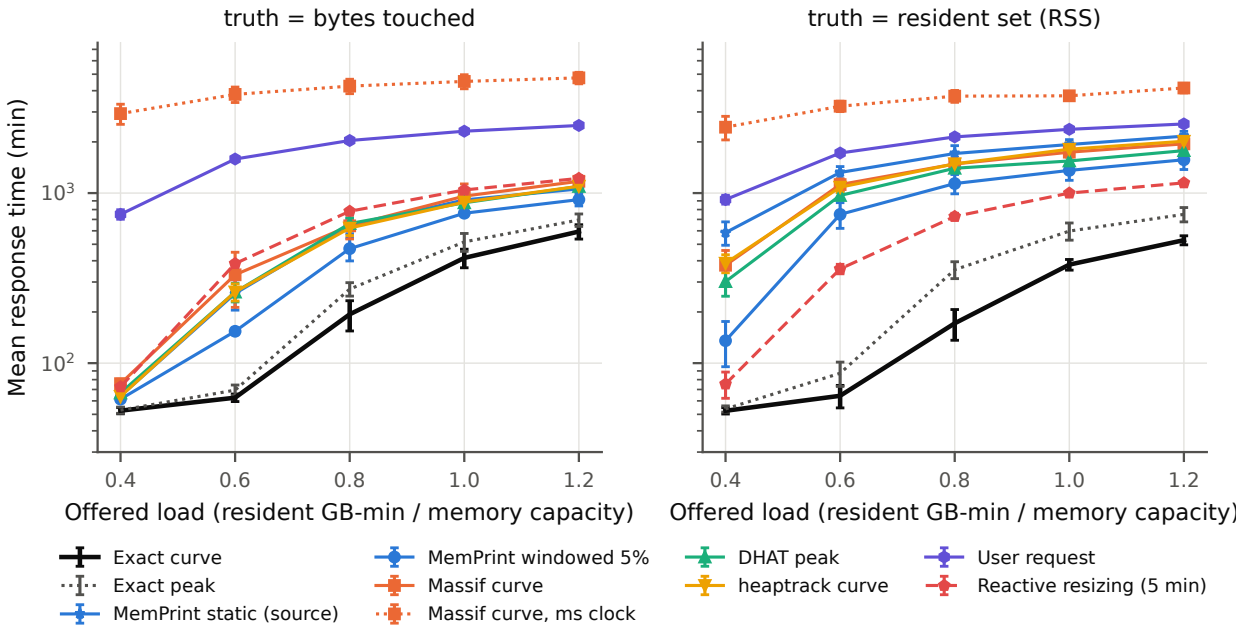


Figure 6: The nine programs the static route predicts (each at its largest input with a prediction): mean response time against offered load, calibrated 90% margins.

log with both fields that we found. A real trace with requested and used memory per job would test whether the effect holds at the memory pressure of a production system.

Scheduler and machine. Jobs use one node, the slowdown is linear in the remote share of the footprint with one β for every job, the pool has no bandwidth contention, and only FCFS with EASY backfilling was tested. The OOM model requeues a killed job with 1.5 times its reservation, and a batch system that killed a job only once, or that swapped instead of killing, would change the kill counts.

Costs. The costs were measured with up to 15 runs sharing the node, so they are slowdowns under load, and Pin's fixed start-up cost weighs heavily on runs of a few seconds. Longer jobs would amortise it.

The static route. The static predictions cover 9 of the 23 programs. They predict a whole-run footprint for C++ programs, so a program that frees and reallocates its data, such as LULESH, is over-predicted, and the interpreter takes up to 5.5 hours for one input of NPB CG.

8 Conclusion

An exact footprint is worth most to a scheduler on disaggregated memory as a curve over the job's run, and the gain comes from letting jobs with peaks at different times share memory. On our corpus and streams, the exact curve gives a 24.6× shorter mean response time than the user's request and needs 5.0 TB where the request needs 11.0 TB. Heap profilers get most peaks right, but their large errors on a few programs, which come from memory that is mapped outside the heap or allocated and never touched, cannot be covered by any margin without over-reserving every

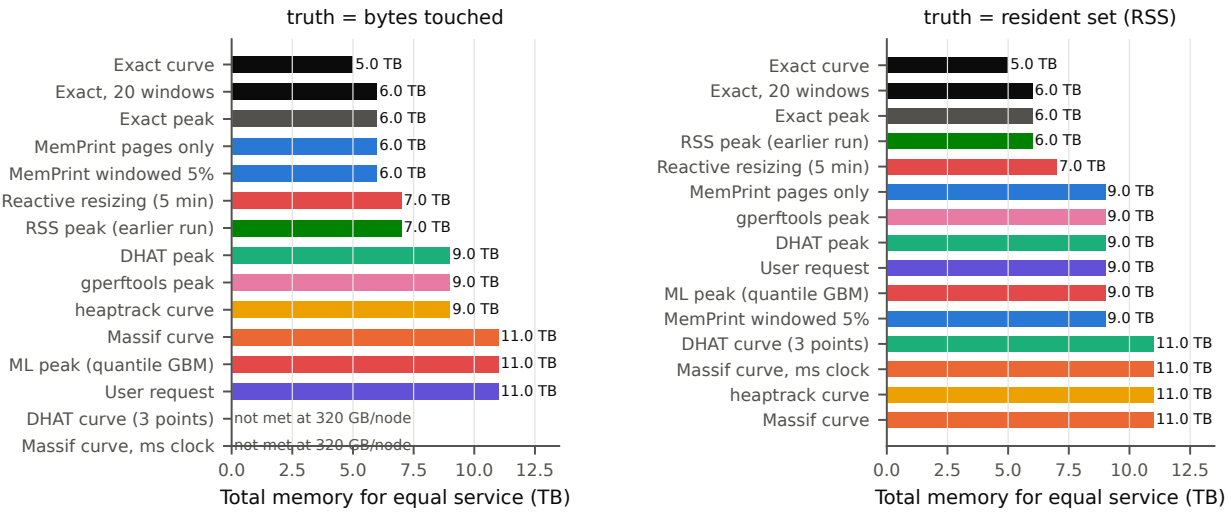


Figure 7: The total memory (32 nodes of local DRAM plus the 1 TB pool) each oracle needs to keep the mean bounded slowdown within 1.5 times that of the exact curve at 128 GB per node, at a fixed arrival rate (load 0.8 on the 128 GB cluster). Local memory per node was swept over 64, 96, 128, 160, 192, 256 and 320 GB.

other job. Among measured estimators, MemPrint's windowed curves come closest to the exact curve, with the smallest margins (2 to 5% for 90% of programs) and 20% more memory than the exact curve for the same service, at 6 to 11 times the native run time. DHAT's three recorded points do not make a usable curve. Two results set the direction of further work. First, under the RSS truth every measured curve loses its advantage because its clock is not wall-clock time, so MemPrint should record the native time of each snapshot and estimate resident pages as well as bytes touched. Second, the static route predicts footprints without running the program, but its errors on programs that free and reallocate memory, and its cost on large inputs, limit it today to programs whose footprint is set by their input sizes.

A Additional figures

References

[1] Emmanuel Amaro et al. Can far memory improve job throughput? In *European Conference on Computer Systems (EuroSys)*, 2020.

[2] Robin Boezennec et al. Allocation strategies for disaggregated memory in HPC systems. In *IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2024.

[3] CXL-ClusterSim: Modeling CXL-based disaggregated memory cluster using gem5 and SST. arXiv:2605.27745, 2026.

[4] CXL-DMSim: A full-system CXL disaggregated memory simulator. arXiv:2411.02282, 2024.

[5] Dror G. Feitelson, Dan Tsafrir, and David Krakov. Parallel workloads archive. `https://www.cs.huji.ac.il/labs/parallel/workload/`. Logs LANL CM5 (LANL-CM5-1994-4.1-cln) and MetaCentrum2 (METACENTRUM-2013-3).

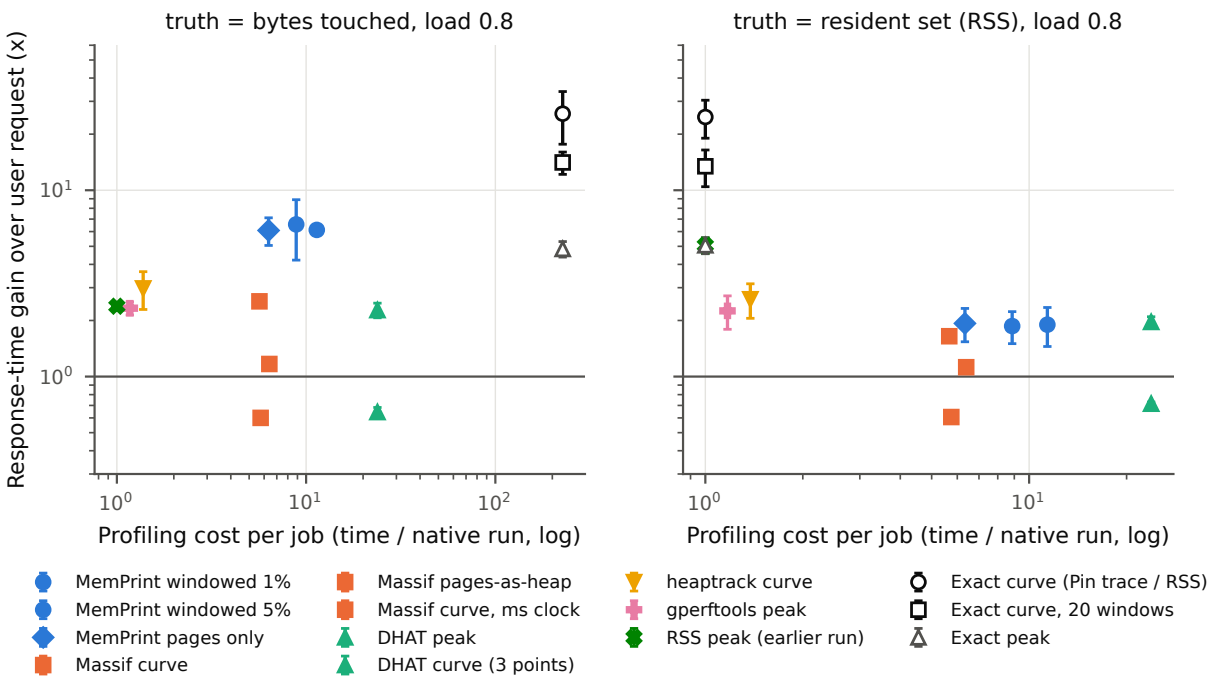


Figure 8: Profiling cost per job (median time over the native run, at the largest inputs) against the gain in mean response time over the user's request, at load 0.8, each oracle at its best margin policy. The exact curve's cost is the full Pin trace under the bytes-touched truth and an RSS sampler under the RSS truth.

[6] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[7] gperftools: TCMalloc and heap profiler. `https://github.com/gperftools/gperftools`. Version 2.18.

[8] KS+: Predicting workflow task memory usage over time. arXiv:2408.12290, 2024.

[9] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[10] David A. Lifka. The ANL/IBM SP scheduling system. Job Scheduling Strategies for Parallel Processing (JSSPP), LNCS 949, 1995.

[11] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[12] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

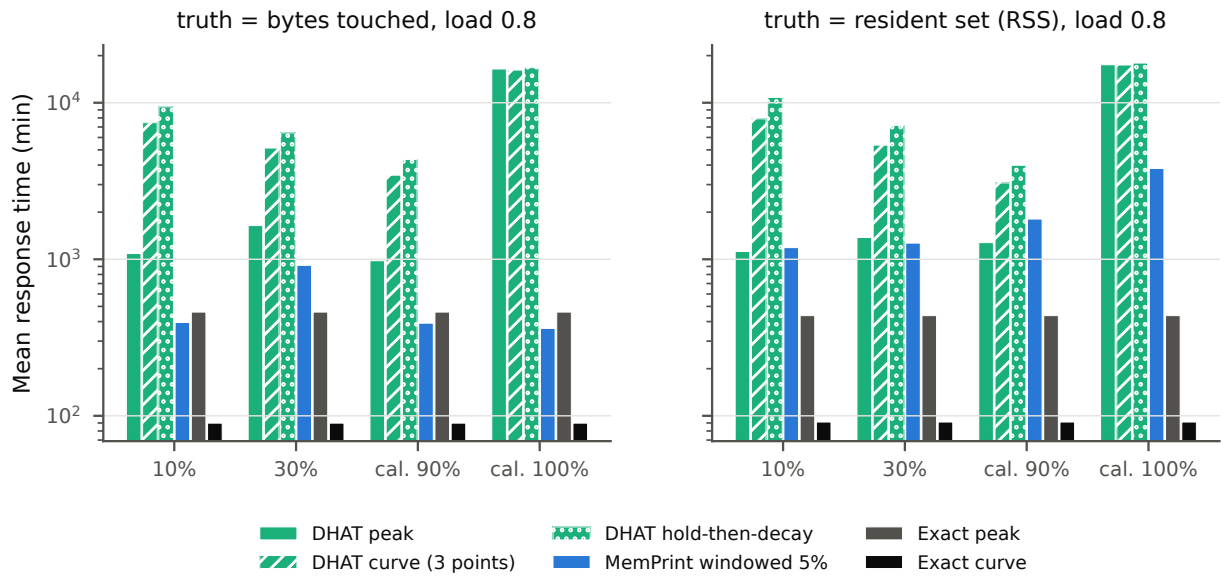


Figure 9: DHAT's peak and the two curves built from its three recorded points, against MemPrint windowed at 5% and the exact peak and curve, at load 0.8 under each margin policy (log scale).

[13] Ivy Peng et al. On the memory underutilization: Exploring disaggregated memory on HPC systems. In *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020.

[14] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. `https://sourceforge.net/projects/polybench/`.

[15] Amit Puri et al. DRackSim: Simulator for rack-scale memory disaggregation. In *ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2024. arXiv:2305.09977.

[16] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at `http://valgrind.org/docs/manual/ms-manual.html`.

[17] Valgrind Developers. *DHAT: a dynamic heap analysis tool*. Available at `https://valgrind.org/docs/manual/dh-manual.html`.

[18] Milian Wolff. heaptrack: a heap memory profiler for Linux. `https://github.com/KDE/heaptrack`. Version 1.3.0.

[19] Yiwei Yang et al. CXLMemSim: A pure software simulated CXL.mem. arXiv:2303.06153, 2023.

[20] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Improving HPC system throughput and response time using memory disaggregation. `https://upcommons.upc.edu/`, 2021.

[21] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance*

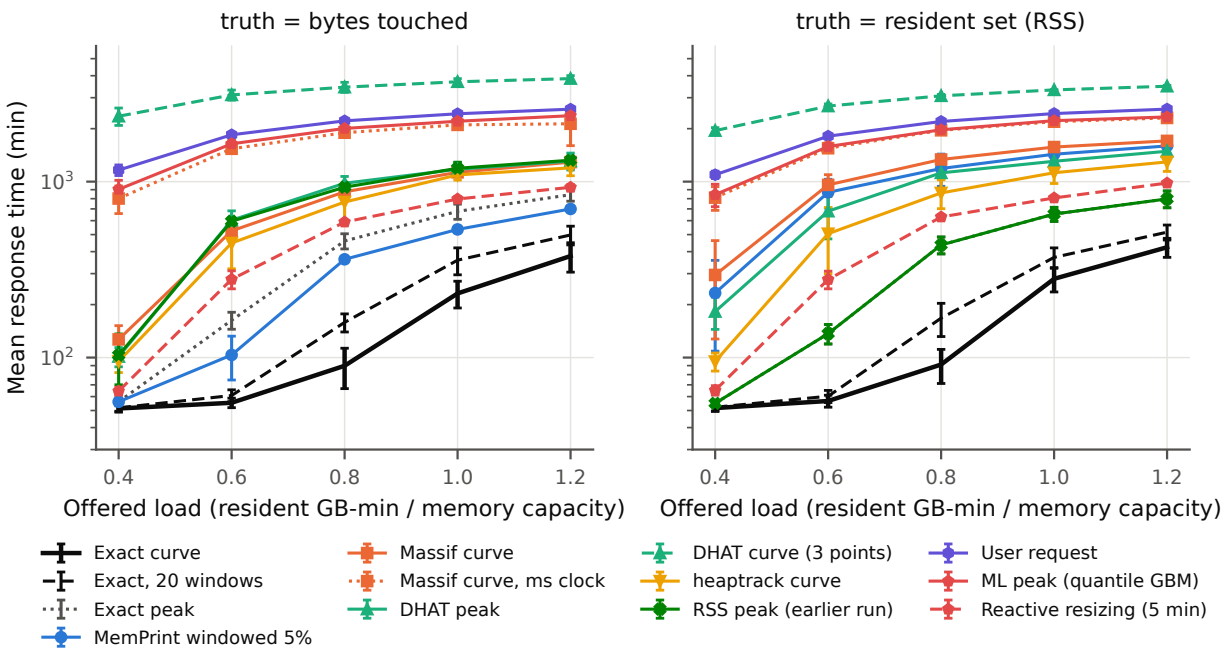


Figure 10: Mean response time against offered load, each oracle at its best margin policy at each load.

Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), 2021.

[22] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.

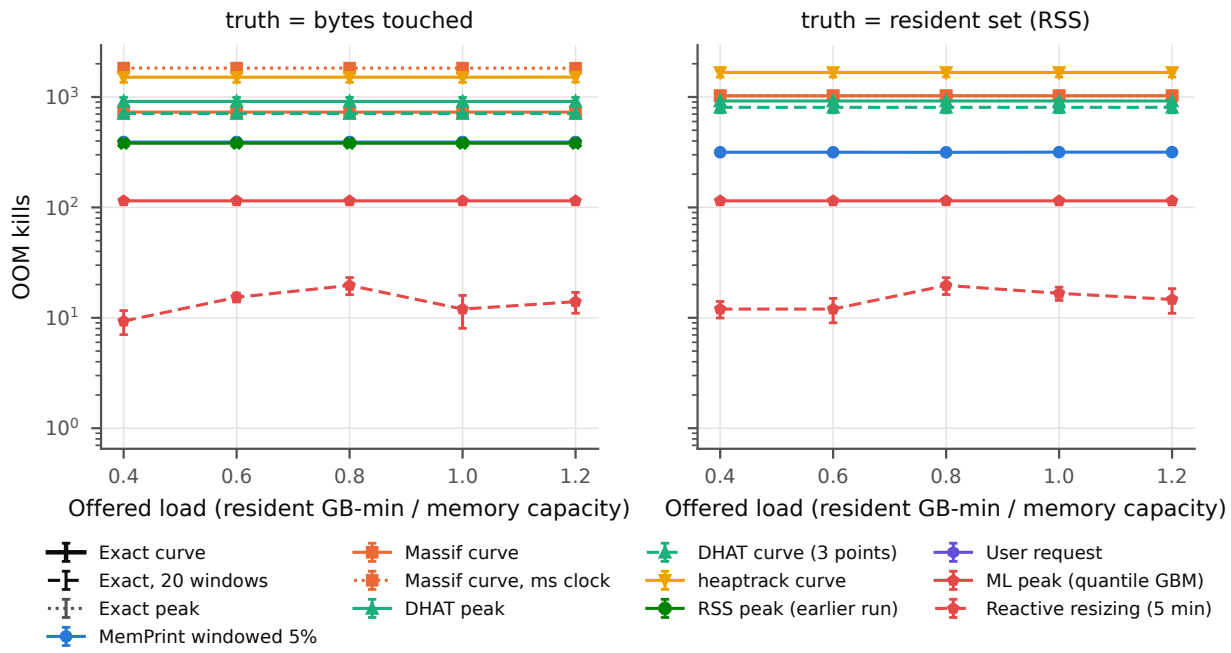


Figure 11: OOM kills per stream of 3,000 jobs against offered load, calibrated 90% margins.